Institución:

Universidad de Costa Rica

Nombre

Steve Puerto Cordero

Carné

C4I552

Tarea Corta #05

Docente

Sleyter Angulo

Fecha de la entrega

8 de oct. de 26.

Readers-Writers

Este problema alude a una situación donde una estructura de datos, sistema o base de datos es leída y modificada por dos hilos que son concurrentes. Entonces, lo que se propone es que mientras se escribe en una de estas estructuras, debemos impedir que se lean o tomen datos de dicha estructura, para evitar leer datos que no son consistentes o que son desactualizados. Entonces la solución es asimétrica, donde puede haber muchos lectores a la vez sobre la sección crítica, hasta que cuando llega un hilo escritor, todos deben salir, y dejarle solo en el cuarto, para que pueda modificar el dato. Esto se describe como exclusión mutua categórica, cada vez que entra un hilo que tiene la categoría "escritor", se excluyen el resto de hilos que están presentes. La primera variable es "readers", que cuenta los lectores que hay en la sala, y un mutex que maneja los cambios que se realizan sobre ese contador. Y hay un roomEmpty que es 1 cuando la sala está vacía.

Solución.

Hilos Escritores: El hilo escritor solo puede entrar si la sala está vacía (roomEmpty). Cuando el hilo escritor sale, la roomEmpty debe quedar en 1 porque no debe quedar nadie ahí.

Hilos Lectores: Pueden actuar como una barrera, el primero que llega, espera la sala, y bloquea a los escritores; mientras que el resto de hilos lectores entran sin esperarlo. El último hilo lector, hace signal; para dejar roomEmpty en 1.

Notas de Adición.

Un hilo escritor podría morir de inanición(falta de trabajo), debido a que los lectores tienen "prioridad", y aunque muchos lleguen después de algún hilo escritor siempre tendrán prioridad cuando un lector esté actuando como barrera.

El patrón Lightswitch.

El código de los lectores se repite mucho, entonces se encapsula en una clase llamada Lightswitch. La idea es la misma que la luz de un cuarto: el primero que entra la prende (bloquea) y el último que sale la apaga (desbloquea). Con esto el código del lector queda más corto y el del escritor no cambia.

```
class Lightswitch:
    def __init__(self):
        self.counter = 0
        self.mutex = Semaphore(1)

    def lock(self, semaphore):
        self.mutex.wait()
        self.counter += 1
        if self.counter == 1:
            semaphore.wait()
        self.mutex.signal()

    def unlock(self, semaphore):
        self.mutex.wait()
        self.counter -= 1
        if self.counter == 0:
            semaphore.signal()
        self.mutex.signal()
```

Ejercicio 1: Readers-writers sin inanición.

Hay que extender la solución para que cuando llegue un escritor, los lectores que ya están dentro terminen, pero que no entren lectores nuevos.

Solución.

```
readSwitch = Lightswitch()
roomEmpty = Semaphore(1)
turnstile = Semaphore(1)

# escritor
turnstile.wait()
roomEmpty.wait()
# sección crítica de escritores
turnstile.signal()
roomEmpty.signal()

# lector
turnstile.wait()
turnstile.signal()
readSwitch.lock(roomEmpty)

# sección crítica de lectores
readSwitch.unlock(roomEmpty)
```

Si llega un escritor y hay lectores adentro, se queda esperando en roomEmpty, pero ya tiene agarrado el turnstile, entonces los lectores nuevos se quedan en el turnstile y no pueden entrar. Cuando sale el último lector se libera roomEmpty y el escritor entra de una vez. Al salir suelta el turnstile y deja pasar al siguiente hilo que esté esperando. Con esto el escritor ya no muere de inanición, aunque el orden de quién sigue depende del planificador.

Ejercicio 2: Readers-writers con prioridad a escritores.

Una vez que llega un escritor, no debe entrar ningún lector hasta que todos los escritores hayan salido.

Solución.

```
readSwitch = Lightswitch()
writeSwitch = Lightswitch()
noReaders = Semaphore(1)
noWriters = Semaphore(1)

# lector
noReaders.wait()
readSwitch.lock(noWriters)
noReaders.signal()
# sección crítica de lectores
readSwitch.unlock(noWriters)

# escritor
writeSwitch.lock(noReaders)
noWriters.wait()

# sección crítica de escritores
noWriters.signal()
writeSwitch.unlock(noReaders)
```

Un lector adentro tiene noWriters, pero no noReaders, entonces un escritor que llega puede agarrar noReaders y los lectores nuevos hacen cola. Cuando sale el último lector se libera noWriters y entra un escritor. Gracias al writeSwitch puede haber varios escritores en cola esperando noWriters mientras noReaders sigue bloqueado, y los lectores solo pueden entrar cuando sale el último escritor. La desventaja es que ahora los que pueden morir de inanición son los lectores.

No Starve-mutex.

La sección anterior trató la inanición categórica. Aquí se trata la inanición de hilos: un hilo espera indefinidamente mientras otros avanzan. En la mayoría de aplicaciones es inaceptable, así que se exige espera acotada (bounded waiting): el tiempo que un hilo espera debe ser demostrablemente finito. Para trabajar sobre esto se tienen 4 propiedades/prioridades:

Propiedad 1	Si solo un hilo está listo para correr, el planificador debe dejarlo correr.
Propiedad 2	Si un hilo está listo, el tiempo hasta que corre está acotado.
Propiedad 3	Si hay hilos esperando en un semáforo cuando alguien hace signal, uno de los que esperan debe despertarse.
Propiedad 4	Si un hilo espera en un semáforo, el número de hilos que se despertarán antes que él está acotado.

Un mutex simple puede tener inanición, si tenemos A,B y C; A tiene el mutex y B y C esperan. Al salir A entra B, pero antes de que B salga A vuelve a la cola junto a C. Cuando B sale no hay garantía de que siga C; si entra A y B vuelve a la cola, se regresa a la situación inicial y el ciclo se repite siempre: C muere de inanición.

Un semáforo que cumple la propiedad 4 se llama semáforo fuerte (por ejemplo, una cola FIFO), y uno que solo cumple la propiedad 3 es un semáforo débil).

Ejercicio: Mutex sin inanición con semáforos débiles.

Hay que escribir una solución de exclusión mutua con semáforos débiles, donde una vez que un hilo llega e intenta entrar, exista un límite de cuántos hilos pueden entrar antes que él. Se asume que hay un número finito de hilos.

Solución (algoritmo de Morris).

```
room1 = room2 = 0
mutex = Semaphore(1)
t1 = Semaphore(1)
t2 = Semaphore(0)

mutex.wait()
room1 += 1
mutex.signal()
```

```
t1.wait()
room2 += 1
mutex.wait()
room1 -= 1

if room1 == 0:
   mutex.signal()
   t2.signal()
else:
   mutex.signal()
   t1.signal()

t2.wait()
room2 -= 1


# sección crítica
if room2 == 0:
   t1.signal()
else:
   t2.signal()
```

Los dos enganches (t1 y t2) dividen el código en tres salas: la sala 1 es antes de t1, la sala 2 es entre t1 y t2, y la sala 3 es la sección crítica. room1 y room2 cuentan cuántos hilos hay en las salas 1 y 2, y mutex protege a room1. Funciona en dos fases. En la primera, t1 está abierto y t2 cerrado, entonces los hilos se registran en la sala 1 y van pasando de uno en uno a la sala 2. Como los hilos son finitos, en algún momento room1 llega a 0, y ahí se abre t2 y t1 queda cerrado. En la segunda fase, t1 está cerrado, o sea que no entran hilos nuevos, y los hilos de la sala 2 pasan uno por uno a la sección crítica. Cuando sale el último (room2 == 0) se cierra t2, se abre t1 y se vuelve al inicio. Así ningún hilo puede dar la vuelta y adelantar a otro, y por eso no hay inanición.

Cigarette smokers

En esta solución participan cuatro hilos: un agente y tres fumadores. Los fumadores repiten para siempre: esperar ingredientes, hacerse el cigarrillo y fumar. Los ingredientes son tabaco, papel y fósforos. El agente tiene suministro infinito de los tres, pero cada fumador tiene suministro infinito de uno solo. El agente elige dos ingredientes distintos al azar y los pone a disposición, el fumador que tiene el complemento que falta los toma, hace el cigarrillo y debe avisarle al agente. El agente de esta analogía es el sistema operativo y los fumadores son las aplicaciones que tienen los recursos necesarios para avanzar.

Se conocen 3 soluciones posibles; en las dos primeras, la premisa es no modificar el agente, pero en la segundo no hay más restricciones. En la trivial(la tercera solución), el agente señala directamente al fumador que debe actuar.

La imposible (de Patil) no deja modificar el código del agente y tampoco permite condicionales ni arreglos de semáforos, y con esas restricciones no tiene solución. La interesante mantiene lo de no modificar al agente pero quita las otras restricciones, y es la que se resuelve aquí. La trivial es la que ya mencioné, donde el agente señala directo al fumador.

Código del agente.

Hay tres agentes (A, B y C) y cada uno pone dos ingredientes distintos. El agentSem hace que el agente espere hasta que un fumador le avise que ya terminó.

```
agentSem = Semaphore(1)
tobacco = Semaphore(0)
paper = Semaphore(0)
match = Semaphore(0)

# Agente A           # Agente B           # Agente C
agentSem.wait()    agentSem.wait()    agentSem.wait()
tobacco.signal()   paper.signal()     tobacco.signal()
paper.signal()     match.signal()     match.signal()
```

La solución que parece natural, pero falla.

Lo primero que uno pensaría es que cada fumador espere directamente sus dos ingredientes. Por ejemplo, el fumador que tiene fósforos:

```
tobacco.wait()
paper.wait()
agentSem.signal()
```

El problema es que se puede dar un deadlock. Si el agente pone tabaco y papel, el fumador con fósforos puede agarrar el tabaco, pero el fumador con tabaco puede agarrar el papel. Entonces el primero se queda esperando papel y el segundo se queda esperando fósforos, y nadie avanza.

Solución con pushers (Parnas).

Se agregan tres hilos auxiliares, los pushers. Ellos responden a las señales del agente, llevan la cuenta de qué "ingredientes" hay en la mesa y despiertan al fumador que corresponde. Las variables son:

```
isTobacco = isPaper = isMatch = False
tobaccoSem = Semaphore(0)
paperSem = Semaphore(0)
matchSem = Semaphore(0)
mutex = Semaphore(1)
```

Pusher A (se despierta cuando hay tabaco en la mesa):

```
tobacco.wait()
mutex.wait()
if isPaper:
   isPaper = False
   matchSem.signal()
elif isMatch:
   isMatch = False
   paperSem.signal()
else:
   isTobacco = True
mutex.signal()
```

Si ya había papel en la mesa, entonces puede proceder el fumador con fósforos; si había fósforos, el fumador con papel; y si no había nada, el pusher solo anota que hay tabaco. Los pushers B y C son iguales, pero cambian los ingredientes que revisan y el semáforo que señalan. Como los pushers hacen todo el trabajo, el código del fumador queda muy simple (este es el fumador con tabaco):

```
tobaccoSem.wait()
makeCigarette()
agentSem.signal()
smoke()
```

Ejercicio: Smokers generalizado.

Parnas propuso una variante más difícil: el agente ya no espera después de poner los ingredientes, entonces se pueden acumular varias unidades del mismo ingrediente en la mesa. Hay que modificar la solución anterior.

Solución.

En vez de booleanos se usan enteros (numTobacco = numPaper = numMatch = 0) y el pusher A queda así:

```
tobacco.wait()
mutex.wait()
if numPaper:
   numPaper -= 1
   matchSem.signal()
elif numMatch:
   numMatch -= 1
   paperSem.signal()
else:
   numTobacco += 1
mutex.signal()
```

Los otros pushers cambian igual. Esto es el patrón scoreboard (marcador): las variables numTobacco, numPaper y numMatch guardan el estado, y cada hilo cuando pasa por el mutex lo revisa y reacciona. Si puede armar un juego completo de ingredientes lo quita de la mesa y despierta al fumador correspondiente, y si no, deja su ingrediente y se va.

Barbershop.

Este problema lo propuso Dijkstra. Hay una barbería con una sala de espera de n sillas y una sala del barbero con la silla de corte. Si no hay clientes el barbero se duerme. Si llega un cliente y todas las sillas están ocupadas, se va. Si el barbero está ocupado pero hay sillas libres, el cliente se sienta, y si el barbero está dormido, el cliente lo despierta. Hay que coordinar al barbero y a los clientes con estas reglas: los clientes llaman a getHairCut; si un cliente llega con la barbería llena llama a balk, que no retorna. El barbero llama a cutHair; y cuando el barbero llama a cutHair tiene que haber exactamente un hilo llamando a getHairCut al mismo tiempo.

Variables.

```
n = 4
customers = 0
mutex = Semaphore(1)
customer = Semaphore(0)
barber = Semaphore(0)
customerDone = Semaphore(0)
barberDone = Semaphore(0)
```

n es el total de clientes que caben en la tienda (tres en la sala de espera y uno en la silla). customers cuenta los clientes que hay y lo protege mutex. Los nombres siguen la idea del rendezvous: customer.wait() quiere decir esperar a un cliente, no que los clientes esperan ahí.

Solución.

Combina un scoreboard y dos rendezvous. Código del cliente:

```
mutex.wait()
if customers == n:
    mutex.signal()
    balk()
customers += 1
mutex.signal()

customer.signal()
barber.wait()

# getHairCut()

customerDone.signal()
barberDone.wait()

mutex.wait()
customers -= 1
mutex.signal()
```

Código del barbero:

```
customer.wait()
barber.signal()

# cutHair()

customerDone.wait()
barberDone.signal()
```

Si ya hay n clientes, el que llega hace balk. Si no, aumenta el contador, avisa con customer.signal() y espera en barber. Cada vez que un cliente avisa, el barbero despierta, hace barber.signal() y da el corte. Si llega otro cliente mientras el barbero está ocupado, en la siguiente vuelta el barbero pasa customer.wait() sin dormirse. El primer rendezvous (customer/barber) sincroniza el inicio del corte, y el segundo (customerDone/barberDone) asegura que el corte terminó antes de que el barbero repita el ciclo y deje pasar al siguiente cliente. Así cutHair y getHairCut pasan al mismo tiempo con un solo cliente.

Referencias Bibliográficas

Downey, A. B. (2016). *The little book of semaphores* (2.ª ed., versión 2.2.1). Green Tea Press. https://greenteapress.com/semaphores/